

ROS2 NAVIGATION ASSIGNMENT

Manual Nav2 Integration

Parv Dixit

ERIC Robotics –Technical Assignment
ROS2 Humble

Overview

This project implements a modular ROS2 Navigation2 (Nav2) stack for the Testbed-T1 robot provided in the assignment repository. Rather than using the standard nav2_bringup package, the navigation pipeline is constructed manually using individual Nav2 plugins and lifecycle nodes.

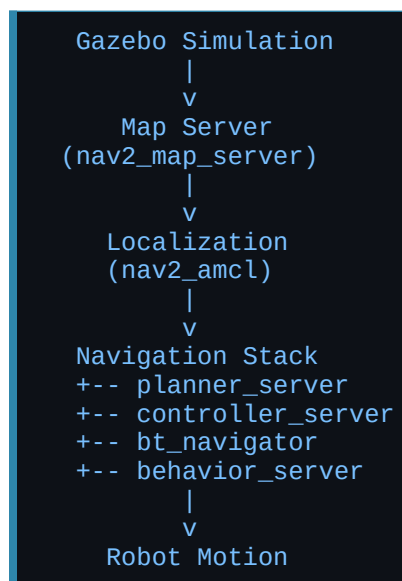
The implementation demonstrates a clear and modular workflow:



System Architecture

The navigation pipeline follows the standard Nav2 architecture. Each stage is implemented as an independent, lifecycle-managed ROS2 node.

All Nav2 components are managed using ROS2 lifecycle nodes and activated through lifecycle managers to ensure deterministic startup of the navigation stack.



Component	Plugin / Node	Function
Map Server	nav2_map_server	Publishes occupancy grid on /map
Localization	nav2_amcl	Estimates robot pose; publishes map → odom
Global Planner	nav2_navfn_planner	Computes global path to goal
Local Controller	nav2_regulated_pure_pursuit_controller	Follows path, handles local obstacles
BT Navigator	nav2_bt_navigator	High-level navigation behavior tree
Recovery	nav2_behaviors	Recovery behaviors on failure

Repository Structure

```
testbed_navigation/  
├── launch/  
│   ├── map_loader.launch.py  
│   ├── localization.launch.py  
│   ├── navigation.launch.py  
│   └── full_navigation.launch.py  
├── config/  
│   ├── amcl_params.yaml  
│   └── nav2_params.yaml  
├── CMakeLists.txt  
├── package.xml  
└── README.md
```

Launch Files

map_loader.launch.py

Launches nav2_map_server with the provided testbed_world.yaml map. Uses a lifecycle manager to configure and activate the map server automatically on startup.

localization.launch.py

Launches nav2_amcl with parameters from amcl_params.yaml. The lifecycle manager activates AMCL automatically. The robot must receive an initial pose estimate via the RViz "2D Pose Estimate" tool before localization begins.

navigation.launch.py

Launches the full Nav2 navigation stack:

1. planner_server — global path planning
2. controller_server — local path following
3. bt_navigator — behavior tree navigation
4. behavior_server — recovery behaviors

full_navigation.launch.py

Unified launch file that starts the entire system in correct sequence: simulation bringup, map loading, localization, and navigation stack.

Running the System

Step 1 — Build the Workspace

```
cd ~/assignment_ws
colcon build
source install/setup.bash
```

Step 2 — Launch Full Navigation Stack

```
ros2 launch testbed_navigation full_navigation.launch.py
```

or

```
# Terminal 1
ros2 launch testbed_bringup testbed_full_bringup.launch.py

# Terminal 2
ros2 launch testbed_navigation map_loader.launch.py

# Terminal 3
ros2 launch testbed_navigation localization.launch.py

# Terminal 4
ros2 launch testbed_navigation navigation.launch.py
```

Step 3 — Navigation Procedure

1. Start the system using the launch command above.
2. In RViz, click 2D Pose Estimate to initialize AMCL localization.
3. Click Nav2 Goal to send a navigation goal.
4. The robot will plan a global path and follow it to the target.

Single-Command Bringup

The entire navigation stack can be launched with one command:

```
ros2 launch testbed_navigation full_navigation.launch.py
```

Nav2 Plugins Used

The following Nav2 plugins were configured for this implementation:

Role	Plugin / Class	Notes
Global Planner	nav2_navfn_planner/ NavfnPlanner	Dijkstra-based global planning
Local Controller	nav2_regulated_pure_pursuit_controller:: RegulatedPurePursuitController	Smooth, obstacle-aware path following
Behavior Tree	navigate_w_replanning_and_recovery.xml	Default Nav2 BT with recovery
Recovery	nav2_behaviors	Spin, backup, wait recovery nodes

Challenges Faced & Solutions

#1 Map Server Lifecycle Issues

Problem: Map server failed to activate due to lifecycle transitions not completing correctly.

Solution: Properly configured the lifecycle manager and ensured map files were installed via CMakeLists.txt.

#2 Map Not Displaying in RViz

Problem: RViz showed 'No map received' even though /map was publishing, due to a QoS mismatch.

Solution: Set the RViz Map display durability policy to Transient Local.

#3 Map File Installation

Problem: The .pgm map image was not being installed into the workspace install directory.

Solution: Added install(DIRECTORY maps DESTINATION share/\${PROJECT_NAME}) to CMakeLists.txt.

#4 AMCL Not Activating Automatically

Problem: AMCL lifecycle node was stuck in the inactive state, preventing initial pose updates.

Solution: Configured lifecycle manager with autostart: true and node_names: ['amcl'].

#5 Missing map → odom Transform

Problem: RViz could not display the map because AMCL had not received an initial pose.

Solution: Use 2D Pose Estimate in RViz to initialize the robot pose before navigation.

#6 Lifecycle Timing Issues

Problem: Occasional activation failures due to node startup ordering.

Solution: Ensured proper launch sequencing and lifecycle configuration ordering.

Testing & Validation

The navigation pipeline was validated using the following test procedure:

#	Test Step	Result
1	Start full navigation system	✓ All nodes activate successfully
2	Initialize robot pose with 2D Pose Estimate	✓ map → odom transform published; particles converge
3	Send Nav2 Goal via RViz	✓ Global path planned and visualized
4	Observe path following	✓ Robot navigates to goal position
5	Confirm goal reached	✓ Success status returned by bt_navigator

Conclusion

This project demonstrates a modular ROS2 Navigation2 setup implemented without using nav2_bringup. The navigation workflow was built from individual Nav2 plugins and managed using lifecycle nodes.

The system successfully performs:

- Map loading using nav2_map_server
- Localization using AMCL with laser scan data
- Autonomous global path planning via NavFn planner
- Local path following using nav2_regulated_pure_pursuit_controller
- Goal navigation via Nav2 Behavior Tree Navigator

The modular structure ensures that each component of the navigation stack can be tested and launched independently, making debugging and future extensions straightforward.

The modular launch architecture allows individual components such as map loading, localization, and navigation to be tested independently before full system integration.

Parv Dixit
parv9457027151@gmail.com